

Cours 4 : Policy Gradients, A2C, PPO, SAC

Approche approfondie : De la théorie à la pratique

Hautot Julien

Polytech

25 novembre 2025

Sommaire

- 1 Introduction : Changer de Paradigme
- 2 Le Policy Gradient Theorem
- 3 REINFORCE & ses Améliorations
- 4 Gestion de l'Exploration : L'Entropie
- 5 Architecture Actor-Critic (A2C)
- 6 Proximal Policy Optimization (PPO)
- 7 Soft Actor-Critic (SAC) : Le Maximum d'Entropie

Rappel : Les méthodes basées sur la Valeur (DQN)

Jusqu'ici, nous utilisons une approche indirecte :

- 1 Apprendre la valeur $Q(s, a)$ (Quelle est la qualité de cet état ?).
- 2 En déduire la politique : $\pi(s) = \arg \max_a Q(s, a)$.

Les limites de cette approche

- **Espaces d'actions continus** : Si a est un angle de moteur (réel continu), calculer $\max_a Q(s, a)$ nécessite une optimisation complexe à chaque pas de temps.
- **Stochasticité** : DQN apprend des politiques déterministes. Or à "Pierre-Feuille-Ciseaux", la meilleure stratégie est d'être aléatoire.
- **Stabilité** : Modifier légèrement Q peut changer radicalement le $\arg \max$ (l'action change brutalement), déstabilisant l'apprentissage.

Rappel : Les méthodes basées sur la Valeur (DQN)

Jusqu'ici, nous utilisons une approche indirecte :

- 1 Apprendre la valeur $Q(s, a)$ (Quelle est la qualité de cet état ?).
- 2 En déduire la politique : $\pi(s) = \arg \max_a Q(s, a)$.

Les limites de cette approche

- **Espaces d'actions continus** : Si a est un angle de moteur (réel continu), calculer $\max_a Q(s, a)$ nécessite une optimisation complexe à chaque pas de temps.
- **Stochasticité** : DQN apprend des politiques déterministes. Or à "Pierre-Feuille-Ciseaux", la meilleure stratégie est d'être aléatoire.
- **Stabilité** : Modifier légèrement Q peut changer radicalement le $\arg \max$ (l'action change brutalement), déstabilisant l'apprentissage.

L'Approche Policy Gradient

Idée : Et si on optimisait directement la politique $\pi_\theta(a|s)$?

π_θ est un réseau de neurones qui prend l'état s en entrée et sort une **distribution de probabilité** sur les actions.

Notre Objectif : Maximiser l'espérance du retour cumulé $J(\theta)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

- τ : Une trajectoire complète $(s_0, a_0, r_1, \dots, s_T)$.
- $R(\tau)$: La somme des récompenses de cette trajectoire.

Challenge : L'environnement est inconnu. Comment calculer le gradient $\nabla_\theta J(\theta)$ pour mettre à jour notre réseau ?

L'Approche Policy Gradient

Idée : Et si on optimisait directement la politique $\pi_\theta(a|s)$?

π_θ est un réseau de neurones qui prend l'état s en entrée et sort une **distribution de probabilité** sur les actions.

Notre Objectif : Maximiser l'espérance du retour cumulé $J(\theta)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

- τ : Une trajectoire complète $(s_0, a_0, r_1, \dots, s_T)$.
- $R(\tau)$: La somme des récompenses de cette trajectoire.

Challenge : L'environnement est inconnu. Comment calculer le gradient $\nabla_\theta J(\theta)$ pour mettre à jour notre réseau ?

Outil Mathématique : Le "Log-Derivative Trick"

Avant de dériver la fonction objectif, rappelons une identité simple de calcul différentiel.

Pour une fonction $x(\theta)$, la dérivée du logarithme est :

$$\nabla \log x = \frac{\nabla x}{x}$$

On peut inverser cette formule pour exprimer le gradient de x :

$$\nabla x = x \cdot \nabla \log x$$

Application à notre politique

$$\nabla_{\theta} \pi_{\theta}(a|s) = \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s)$$

Pourquoi ? Cela nous permettra de faire réapparaître la probabilité π pour former une Espérance mathématique.

Outil Mathématique : Le "Log-Derivative Trick"

Avant de dériver la fonction objectif, rappelons une identité simple de calcul différentiel.

Pour une fonction $x(\theta)$, la dérivée du logarithme est :

$$\nabla \log x = \frac{\nabla x}{x}$$

On peut inverser cette formule pour exprimer le gradient de x :

$$\nabla x = x \cdot \nabla \log x$$

Application à notre politique

$$\nabla_{\theta} \pi_{\theta}(a|s) = \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s)$$

Pourquoi ? Cela nous permettra de faire réapparaître la probabilité π pour former une Espérance mathématique.

Probabilité d'une trajectoire

Quelle est la probabilité $P(\tau|\theta)$ qu'une trajectoire se produise ?

$$P(\tau|\theta) = \underbrace{\rho(s_0)}_{\text{Départ}} \prod_{t=0}^{T-1} \underbrace{\pi_{\theta}(a_t|s_t)}_{\text{Politique (Nous)}} \underbrace{P(s_{t+1}|s_t, a_t)}_{\text{Dynamique (Env.)}}$$

Prenons le logarithme de cette expression :

$$\log P(\tau|\theta) = \log \rho(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t|s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1}|s_t, a_t)$$

Observation Fondamentale

Si on dérive par rapport à θ (∇_{θ}), les termes $\rho(s_0)$ et $P(s'|s, a)$ s'annulent car ils ne dépendent pas des poids du réseau !

$$\nabla_{\theta} \log P(\tau|\theta) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$$

Probabilité d'une trajectoire

Quelle est la probabilité $P(\tau|\theta)$ qu'une trajectoire se produise ?

$$P(\tau|\theta) = \underbrace{\rho(s_0)}_{\text{Départ}} \prod_{t=0}^{T-1} \underbrace{\pi_{\theta}(a_t|s_t)}_{\text{Politique (Nous)}} \underbrace{P(s_{t+1}|s_t, a_t)}_{\text{Dynamique (Env.)}}$$

Prenons le logarithme de cette expression :

$$\log P(\tau|\theta) = \log \rho(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t|s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1}|s_t, a_t)$$

Observation Fondamentale

Si on dérive par rapport à θ (∇_{θ}), les termes $\rho(s_0)$ et $P(s'|s, a)$ s'annulent car ils ne dépendent pas des poids du réseau !

$$\nabla_{\theta} \log P(\tau|\theta) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$$

Probabilité d'une trajectoire

Quelle est la probabilité $P(\tau|\theta)$ qu'une trajectoire se produise ?

$$P(\tau|\theta) = \underbrace{\rho(s_0)}_{\text{Départ}} \prod_{t=0}^{T-1} \underbrace{\pi_{\theta}(a_t|s_t)}_{\text{Politique (Nous)}} \underbrace{P(s_{t+1}|s_t, a_t)}_{\text{Dynamique (Env.)}}$$

Prenons le logarithme de cette expression :

$$\log P(\tau|\theta) = \log \rho(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t|s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1}|s_t, a_t)$$

Observation Fondamentale

Si on dérive par rapport à θ (∇_{θ}), les termes $\rho(s_0)$ et $P(s'|s, a)$ s'annulent car ils ne dépendent pas des poids du réseau !

$$\nabla_{\theta} \log P(\tau|\theta) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$$

Le Théorème du Gradient de la Politique

Nous voulons $\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}[R(\tau)]$. Par définition de l'espérance continue (intégrale) :

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int P(\tau|\theta) R(\tau) d\tau$$

On rentre le gradient dans l'intégrale et on utilise le *Log-Trick* ($\nabla_{\theta} P = P \nabla_{\theta} \log P$) :

$$= \int P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau) d\tau$$

On reconnaît la définition de l'Espérance ($\int P(x) f(x) dx = \mathbb{E}[f(x)]$) :

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)]$$

Formule Finale (Policy Gradient Theorem)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(\tau) \right]$$

Le Théorème du Gradient de la Politique

Nous voulons $\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}[R(\tau)]$. Par définition de l'espérance continue (intégrale) :

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int P(\tau|\theta) R(\tau) d\tau$$

On rentre le gradient dans l'intégrale et on utilise le *Log-Trick* ($\nabla_{\theta} P = P \nabla_{\theta} \log P$) :

$$= \int P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau) d\tau$$

On reconnaît la définition de l'Espérance ($\int P(x) f(x) dx = \mathbb{E}[f(x)]$) :

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)]$$

Formule Finale (Policy Gradient Theorem)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(\tau) \right]$$

Le Théorème du Gradient de la Politique

Nous voulons $\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}[R(\tau)]$. Par définition de l'espérance continue (intégrale) :

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int P(\tau|\theta) R(\tau) d\tau$$

On rentre le gradient dans l'intégrale et on utilise le *Log-Trick* ($\nabla_{\theta} P = P \nabla_{\theta} \log P$) :

$$= \int P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau) d\tau$$

On reconnaît la définition de l'Espérance ($\int P(x) f(x) dx = \mathbb{E}[f(x)]$) :

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)]$$

Formule Finale (Policy Gradient Theorem)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(\tau) \right]$$

Le Théorème du Gradient de la Politique

Nous voulons $\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}[R(\tau)]$. Par définition de l'espérance continue (intégrale) :

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int P(\tau|\theta) R(\tau) d\tau$$

On rentre le gradient dans l'intégrale et on utilise le *Log-Trick* ($\nabla_{\theta} P = P \nabla_{\theta} \log P$) :

$$= \int P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau) d\tau$$

On reconnaît la définition de l'Espérance ($\int P(x) f(x) dx = \mathbb{E}[f(x)]$) :

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)]$$

Formule Finale (Policy Gradient Theorem)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(\tau) \right]$$

Intuition de REINFORCE

La formule nous dit :

$$\Delta\theta \propto \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot R(\tau)$$

- **Si $R(\tau)$ est élevé (bonne trajectoire)** : On déplace θ pour **augmenter** la probabilité $\pi(a|s)$ de refaire ces actions.
- **Si $R(\tau)$ est faible/négatif** : On déplace θ pour **diminuer** la probabilité de refaire ces actions.

C'est du "Trial-and-Error" formalisé mathématiquement.

Problème 1 : La Causalité

Dans la version naïve, on utilise $R(\tau)$ (le score total) pour juger chaque action.

Exemple : J'ai bien joué, puis j'ai fait une erreur à la fin et perdu.

L'algorithme va punir **toutes** les actions, même les bonnes du début.

Solution : "Rewards-to-go" Une action à t ne doit être jugée que sur les récompenses futures.

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

$$\nabla_{\theta} J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

Problème 1 : La Causalité

Dans la version naïve, on utilise $R(\tau)$ (le score total) pour juger chaque action.

Exemple : J'ai bien joué, puis j'ai fait une erreur à la fin et perdu.

L'algorithme va punir **toutes** les actions, même les bonnes du début.

Solution : "Rewards-to-go" Une action à t ne doit être jugée que sur les récompenses futures.

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$
$$\nabla_{\theta} J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

Problème 2 : La Variance

REINFORCE est un algorithme Monte-Carlo : on utilise *une seule* trajectoire pour estimer le gradient.

- Selon la chance (stochasticité), deux épisodes peuvent donner des retours très différents.
- Le gradient devient bruité : l'apprentissage oscille ("Random Walk").

Solution : La Baseline $b(s)$ Au lieu de regarder la valeur brute G_t , on regarde si G_t est meilleur que la moyenne.

$$\delta_t = G_t - b(s_t)$$

Généralement, $b(s_t) \approx V(s_t)$ (Valeur moyenne de l'état).

Problème 2 : La Variance

REINFORCE est un algorithme Monte-Carlo : on utilise *une seule* trajectoire pour estimer le gradient.

- Selon la chance (stochasticité), deux épisodes peuvent donner des retours très différents.
- Le gradient devient bruité : l'apprentissage oscille ("Random Walk").

Solution : La Baseline $b(s)$ Au lieu de regarder la valeur brute G_t , on regarde si G_t est **meilleur que la moyenne**.

$$\delta_t = G_t - b(s_t)$$

Généralement, $b(s_t) \approx V(s_t)$ (Valeur moyenne de l'état).

Algorithme REINFORCE avec Baseline

Initialiser π_θ et V_ϕ (Baseline);

for *chaque épisode* **do**

1. Générer une trajectoire complète τ ;
2. Calculer les retours G_t pour chaque pas t ;
3. Calculer l'avantage $\delta_t = G_t - V_\phi(s_t)$;
4. Mettre à jour la Politique θ (montée de gradient) :

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta \log \pi_\theta(a_t | s_t)$$

5. Mettre à jour la Baseline ϕ (descente de gradient MSE) :

$$\phi \leftarrow \phi - \beta \nabla_\phi (G_t - V_\phi(s_t))^2$$

end

Algorithm 1: REINFORCE avec Baseline

Le Problème de la Convergence Prématurée

En Policy Gradient, la politique peut devenir déterministe trop vite ($\pi(a|s) \rightarrow 1$ pour une action sub-optimale).

- Une fois que la probabilité d'une action est proche de 0, elle n'est plus jamais échantillonnée.
- L'agent arrête d'explorer.

Solution : Régularisation par l'Entropie de Shannon L'entropie H mesure "l'incertitude" ou le caractère aléatoire d'une distribution :

$$H(\pi(\cdot|s_t)) = - \sum_a \pi(a|s_t) \log \pi(a|s_t)$$

Le Problème de la Convergence Prématurée

En Policy Gradient, la politique peut devenir déterministe trop vite ($\pi(a|s) \rightarrow 1$ pour une action sub-optimale).

- Une fois que la probabilité d'une action est proche de 0, elle n'est plus jamais échantillonnée.
- L'agent arrête d'explorer.

Solution : Régularisation par l'Entropie de Shannon L'entropie H mesure "l'incertitude" ou le caractère aléatoire d'une distribution :

$$H(\pi(\cdot|s_t)) = - \sum_a \pi(a|s_t) \log \pi(a|s_t)$$

Intégration dans la Fonction Objectif

On modifie la fonction objectif pour encourager l'entropie :

$$J_{aug}(\theta) = J(\theta) + \beta \cdot \mathbb{E}_t[H(\pi(\cdot|s_t))]$$

- β : Coefficient d'entropie (hyperparamètre).
- **Effet** : Pousse les probabilités $\pi(a|s)$ à rester "étalées" (proches de l'uniforme) au début.
- Empêche le "Collapsing" de la politique.

Note : Dans les implémentations modernes (A2C/PPO), on soustrait l'entropie à la Loss ($Loss = PolicyLoss - \beta H$).

Le concept Actor-Critic

Pour réduire la variance de REINFORCE, nous avons introduit une Baseline $V(s)$. Dans A2C, nous apprenons cette Baseline via un second réseau de neurones.

Deux composantes :

Le Critique (Critic) $V_{\phi}(s)$

Estime la valeur de l'état. Sert de Baseline pour réduire la variance.

L'Acteur (Actor) $\pi_{\theta}(a|s)$

Choisit l'action. Mise à jour dans la direction suggérée par le Critique.

La Fonction Avantage

Comment le Critique guide-t-il l'Acteur ? Via l'Avantage $A(s, a)$. L'avantage répond à : *"Cette action était-elle meilleure que ce à quoi je m'attendais en moyenne ?"*

$$A(s_t, a_t) = \underbrace{Q_w(s_t, a_t)}_{\text{Valeur réelle action}} - \underbrace{V_\phi(s_t)}_{\text{Valeur moyenne état}}$$

En pratique, on estime Q via bootstrapping (TD Error) :

$$A(s_t, a_t) \approx \underbrace{r_{t+1} + \gamma V_\phi(s_{t+1})}_{\text{TD Target}} - V_\phi(s_t)$$

La Loss Totale A2C

Dans A2C, on optimise souvent un réseau partagé (Shared Backbone). La fonction de perte totale à minimiser est composite :

$$L_{total} = L_{actor} + c_1 L_{critic} - c_2 H$$

- ❶ **Actor Loss** : $-\log \pi(a_t | s_t) \cdot A(s_t, a_t)$. (On veut maximiser l'avantage).
- ❷ **Critic Loss** : $(A(s_t, a_t))^2$ ou MSE. (On veut que $V(s)$ soit juste).
- ❸ **Entropy Bonus** : $-H(\pi)$. (On veut maximiser l'entropie, donc minimiser son négatif).

Motivation : La "Trust Region"

Le gradient policy standard est instable. Un pas trop grand (α trop élevé) peut détruire la politique.

Idée : On veut maximiser le gain, mais en s'assurant que la nouvelle politique π_θ reste "proche" de l'ancienne $\pi_{\theta_{old}}$.

On définit le ratio de probabilité :

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

Si $r_t(\theta) > 1$, l'action est plus probable qu'avant.

Le "Clipped Surrogate Objective"

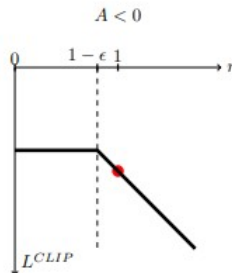
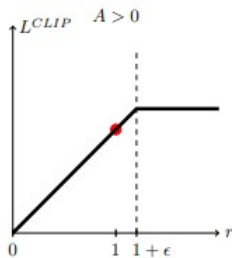
PPO introduit une fonction objectif astucieuse qui "coupe" (clip) le gradient si le changement est trop brutal.

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

Analyse des cas ($\epsilon = 0.2$) :

- **Avantage positif ($A > 0$)** : On veut augmenter la proba. Mais si $r_t > 1.2$, le terme est clippé. Le gradient devient nul $\rightarrow$ on arrête de pousser les poids.
- **Avantage négatif ($A < 0$)** : On veut diminuer la proba. Si $r_t < 0.8$, on clippe. On évite de réduire la probabilité à 0 instantanément.

Visualisation du Clipping



Si $A > 0$: La courbe monte linéairement puis devient plate (gradient nul) quand le ratio dépasse $1 + \epsilon$.

Si $A < 0$: La courbe descend puis devient plate quand le ratio passe sous $1 - \epsilon$.

Cela crée une **"Trust Region"** implicite sans calculs matriciels complexes (contrairement à TRPO).

Algorithme PPO Complet

for *itération* = 1, 2, ... **do**

1. Lancer l'ancienne politique $\pi_{\theta_{old}}$ pendant T pas (Collecte de données).;
2. Calculer les avantages $\hat{A}_t$ (souvent avec GAE).;
3. Optimiser L_{total} sur K époques (minibatches) :

$$L = L^{CLIP}(\theta) - c_1(V_\phi(s) - V_{target})^2 + c_2 H(\pi)$$

4. $\theta_{old} \leftarrow \theta$;

end

Algorithm 2: PPO

PPO est aujourd'hui l'algorithme par défaut en RL (robotique, LLM RLHF, jeux vidéo).

Résumé et Conclusion

Méthode	Concept Clé	Stabilité
Policy Gradient	Optimiser π directement	-
REINFORCE	Utiliser Monte-Carlo (G_t)	Faible (Variance)
Baseline	Soustraire la moyenne ($G_t - V$)	Moyenne
A2C	Utiliser un Critique (TD)	Bonne
PPO	Limiter le changement (ϵ)	Excellente

Prochaine étape : Implémentation pratique de PPO sur un environnement Gymnasium.

SAC : Soft Actor-Critic

SAC est un des algorithmes de contrôle continu les plus efficaces en termes d'échantillons (sample efficiency).

Philosophie Fondamentale : Le Maximum d'Entropie (MaxEnt).

- Traditionnellement, le RL maximise uniquement la récompense : $\pi^* = \arg \max_{\pi} \mathbb{E}[R]$.
- SAC cherche la politique qui maximise à la fois la récompense **ET** l'aléa (l'exploration).

Nouvelle Fonction Objectif "Soft"

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T (r_t + \alpha H(\pi(\cdot | s_t))) \right]$$

- r_t : Récompense classique.
- $H(\pi)$: Terme d'Entropie (encourage l'exploration).
- α : **Coefficient de Température** (Hyperparamètre clé). Il contrôle la primauté de l'exploration par rapport à la récompense.

SAC : Soft Actor-Critic

SAC est un des algorithmes de contrôle continu les plus efficaces en termes d'échantillons (sample efficiency).

Philosophie Fondamentale : Le Maximum d'Entropie (MaxEnt).

- Traditionnellement, le RL maximise uniquement la récompense :
 $\pi^* = \arg \max_{\pi} \mathbb{E}[R]$.
- SAC cherche la politique qui maximise à la fois la récompense **ET** l'aléa (l'exploration).

Nouvelle Fonction Objectif "Soft"

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T (r_t + \alpha H(\pi(\cdot|s_t))) \right]$$

- r_t : Récompense classique.
- $H(\pi)$: Terme d'Entropie (encourage l'exploration).
- α : **Coefficient de Température** (Hyperparamètre clé). Il contrôle la primauté de l'exploration par rapport à la récompense.

SAC : Soft Actor-Critic

SAC est un des algorithmes de contrôle continu les plus efficaces en termes d'échantillons (sample efficiency).

Philosophie Fondamentale : Le Maximum d'Entropie (MaxEnt).

- Traditionnellement, le RL maximise uniquement la récompense :
 $\pi^* = \arg \max_{\pi} \mathbb{E}[R]$.
- SAC cherche la politique qui maximise à la fois la récompense **ET** l'aléa (l'exploration).

Nouvelle Fonction Objectif "Soft"

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T (r_t + \alpha H(\pi(\cdot | s_t))) \right]$$

- r_t : Récompense classique.
- $H(\pi)$: Terme d'Entropie (encourage l'exploration).
- α : **Coefficient de Température** (Hyperparamètre clé). Il contrôle la primauté de l'exploration par rapport à la récompense.

SAC : Soft Actor-Critic

SAC est un des algorithmes de contrôle continu les plus efficaces en termes d'échantillons (sample efficiency).

Philosophie Fondamentale : Le Maximum d'Entropie (MaxEnt).

- Traditionnellement, le RL maximise uniquement la récompense :
 $\pi^* = \arg \max_{\pi} \mathbb{E}[R]$.
- SAC cherche la politique qui maximise à la fois la récompense **ET** l'aléa (l'exploration).

Nouvelle Fonction Objectif "Soft"

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T (r_t + \alpha H(\pi(\cdot | s_t))) \right]$$

- r_t : Récompense classique.
- $H(\pi)$: Terme d'Entropie (encourage l'exploration).
- α : **Coefficient de Température** (Hyperparamètre clé). Il contrôle la primauté de l'exploration par rapport à la récompense.

Architecture et Mécanisme Off-Policy

SAC utilise une architecture plus complexe mais bénéficie de l'efficacité **Off-Policy**.

1. Off-Policy et Réutilisation des Données

- L'agent stocke les transitions (s, a, r, s') dans un grand **Replay Buffer**.
- Il s'entraîne en tirant aléatoirement des lots de ce buffer, indépendamment de la politique qui les a générés.
- **Résultat** : Réduit considérablement le nombre d'interactions nécessaires avec l'environnement (haute efficacité d'échantillon).

2. Réseaux Multiples L'implémentation standard utilise généralement **quatre réseaux** (ou six) :

- $\pi_{\theta}(a|s)$: Le réseau **Acteur** (Politique stochastique).
- $Q_{\phi_1}(s, a)$ et $Q_{\phi_2}(s, a)$: Deux réseaux **Critiques** (pour réduire le sur-apprentissage).
- Q'_{ϕ_1} et Q'_{ϕ_2} : Deux réseaux **Cibles** (Target Networks) pour le calcul des valeurs futures.

Architecture et Mécanisme Off-Policy

SAC utilise une architecture plus complexe mais bénéficie de l'efficacité **Off-Policy**.

1. Off-Policy et Réutilisation des Données

- L'agent stocke les transitions (s, a, r, s') dans un grand **Replay Buffer**.
- Il s'entraîne en tirant aléatoirement des lots de ce buffer, indépendamment de la politique qui les a générés.
- **Résultat** : Réduit considérablement le nombre d'interactions nécessaires avec l'environnement (haute efficacité d'échantillon).

2. **Réseaux Multiples** L'implémentation standard utilise généralement **quatre réseaux** (ou six) :

- $\pi_{\theta}(a|s)$: Le réseau **Acteur** (Politique stochastique).
- $Q_{\phi_1}(s, a)$ et $Q_{\phi_2}(s, a)$: Deux réseaux **Critiques** (pour réduire le sur-apprentissage).
- Q'_{ϕ_1} et Q'_{ϕ_2} : Deux réseaux **Cibles** (Target Networks) pour le calcul des valeurs futures.

Architecture et Mécanisme Off-Policy

SAC utilise une architecture plus complexe mais bénéficie de l'efficacité **Off-Policy**.

1. Off-Policy et Réutilisation des Données

- L'agent stocke les transitions (s, a, r, s') dans un grand **Replay Buffer**.
- Il s'entraîne en tirant aléatoirement des lots de ce buffer, indépendamment de la politique qui les a générés.
- **Résultat** : Réduit considérablement le nombre d'interactions nécessaires avec l'environnement (haute efficacité d'échantillon).

2. Réseaux Multiples L'implémentation standard utilise généralement **quatre réseaux** (ou six) :

- $\pi_{\theta}(a|s)$: Le réseau **Acteur** (Politique stochastique).
- $Q_{\phi_1}(s, a)$ et $Q_{\phi_2}(s, a)$: Deux réseaux **Critiques** (pour réduire le sur-apprentissage).
- $Q'_{\phi'_1}$ et $Q'_{\phi'_2}$: Deux réseaux **Cibles** (Target Networks) pour le calcul des valeurs futures.

Les Trois Mises à Jour de SAC

Soft Actor-Critic optimise **trois pertes différentes** : les deux Critiques, l'Acteur, et la Température.

1. Perte des Critiques (Minimisation)

Les Critiques apprennent la **Soft Q-Value** en minimisant :

$$L_Q(\phi_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} \left[(Q_{\phi_i}(s, a) - y)^2 \right]$$

avec la cible soft (Soft Bellman Target) :

$$y = r + \gamma \left[\min_i Q'_{\phi'_i}(s', a') - \alpha \log \pi_{\theta}(a'|s') \right]$$

- a' est échantillonné depuis la politique actuelle $\pi_{\theta}(\cdot|s')$.
- Le minimum des deux Critiques réduit le *biais optimiste*.
- Le terme $-\alpha \log \pi$ incorpore l'entropie dans la valeur future.

1. Perte des Critiques (Minimisation)

Les Critiques apprennent la **Soft Q-Value** en minimisant :

$$L_Q(\phi_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} \left[(Q_{\phi_i}(s, a) - y)^2 \right]$$

avec la cible soft (Soft Bellman Target) :

$$y = r + \gamma \left[\min_i Q'_{\phi'_i}(s', a') - \alpha \log \pi_{\theta}(a'|s') \right]$$

- a' est échantillonné depuis la politique actuelle $\pi_{\theta}(\cdot|s')$.
- Le minimum des deux Critiques réduit le *biais optimiste*.
- Le terme $-\alpha \log \pi$ incorpore l'entropie dans la valeur future.

2. Perte de l'Acteur (Minimisation)

L'Acteur est mis à jour pour équilibrer récompense et entropie :

$$L_{\pi}(\theta) = \mathbb{E}_{s \sim \mathcal{B}, a \sim \pi} \left[\alpha \log \pi_{\theta}(a|s) - \min_i Q_{\phi_i}(s, a) \right]$$

- L'acteur cherche des actions à haute Q-value.
- Mais il garde une politique diversifiée grâce au terme d'entropie.

La Température α : Ajustement Automatique

Contrairement à PPO ou A2C, SAC **apprend automatiquement** le coefficient d'entropie α .

Intuition :

- α grand $\Rightarrow$ politique très aléatoire (exploration).
- α petit $\Rightarrow$ politique plus déterministe (exploitation).

SAC maintient une entropie cible H_{cible} :

$$H_{\text{cible}} = -\dim(\text{action space})$$

Perte associée à α :

$$L_{\alpha} = \mathbb{E}_{a \sim \pi(\cdot|s)} [-\alpha (\log \pi(a|s) + H_{\text{cible}})]$$

- Si l'entropie réelle est **plus basse** que la cible : $\Rightarrow \alpha$ augmente (plus d'exploration).
- Si l'entropie est **trop haute** : $\Rightarrow \alpha$ diminue (moins d'exploration).

La Température α : Ajustement Automatique

Contrairement à PPO ou A2C, SAC **apprend automatiquement** le coefficient d'entropie α .

Intuition :

- α grand $\Rightarrow$ politique très aléatoire (exploration).
- α petit $\Rightarrow$ politique plus déterministe (exploitation).

SAC maintient une entropie cible H_{cible} :

$$H_{\text{cible}} = -\dim(\text{action space})$$

Perte associée à α :

$$L_{\alpha} = \mathbb{E}_{a \sim \pi(\cdot|s)} [-\alpha (\log \pi(a|s) + H_{\text{cible}})]$$

- Si l'entropie réelle est **plus basse** que la cible : $\Rightarrow \alpha$ augmente (plus d'exploration).
- Si l'entropie est **trop haute** : $\Rightarrow \alpha$ diminue (moins d'exploration).

La Température α : Ajustement Automatique

Contrairement à PPO ou A2C, SAC **apprend automatiquement** le coefficient d'entropie α .

Intuition :

- α grand $\Rightarrow$ politique très aléatoire (exploration).
- α petit $\Rightarrow$ politique plus déterministe (exploitation).

SAC maintient une entropie cible H_{cible} :

$$H_{\text{cible}} = -\dim(\text{action space})$$

Perte associée à α :

$$L_{\alpha} = \mathbb{E}_{a \sim \pi(\cdot|s)} [-\alpha (\log \pi(a|s) + H_{\text{cible}})]$$

- Si l'entropie réelle est **plus basse** que la cible : $\Rightarrow \alpha$ augmente (plus d'exploration).
- Si l'entropie est **trop haute** : $\Rightarrow \alpha$ diminue (moins d'exploration).

La Température α : Ajustement Automatique

Contrairement à PPO ou A2C, SAC **apprend automatiquement** le coefficient d'entropie α .

Intuition :

- α grand $\Rightarrow$ politique très aléatoire (exploration).
- α petit $\Rightarrow$ politique plus déterministe (exploitation).

SAC maintient une entropie cible H_{cible} :

$$H_{\text{cible}} = -\dim(\text{action space})$$

Perte associée à α :

$$L_{\alpha} = \mathbb{E}_{a \sim \pi(\cdot|s)} [-\alpha (\log \pi(a|s) + H_{\text{cible}})]$$

- Si l'entropie réelle est **plus basse** que la cible : $\Rightarrow \alpha$ augmente (plus d'exploration).
- Si l'entropie est **trop haute** : $\Rightarrow \alpha$ diminue (moins d'exploration).

La Température α : Ajustement Automatique

Contrairement à PPO ou A2C, SAC **apprend automatiquement** le coefficient d'entropie α .

Intuition :

- α grand $\Rightarrow$ politique très aléatoire (exploration).
- α petit $\Rightarrow$ politique plus déterministe (exploitation).

SAC maintient une entropie cible H_{cible} :

$$H_{\text{cible}} = -\dim(\text{action space})$$

Perte associée à α :

$$L_{\alpha} = \mathbb{E}_{a \sim \pi(\cdot|s)} [-\alpha (\log \pi(a|s) + H_{\text{cible}})]$$

- Si l'entropie réelle est **plus basse** que la cible : $\Rightarrow \alpha$ augmente (plus d'exploration).
- Si l'entropie est **trop haute** : $\Rightarrow \alpha$ diminue (moins d'exploration).